CAPSTONE PROJECT

CUSTOMER ENGAGEMENT & SERVICE ANALYTICS PLATFORM

DANDAVENI GANESH

WIPRO NGA - .Net React (MS Dynamics) - FY26 - C3

Instructor Name : Parth Shukla

Table of Contents:

SNo.	Contents	Page No.
1.	Introduction	3
2.	Objectives	3
3.	System Architecture	4
4.	Data Flow Diagram (DFD)	5
5.	Database Design (ER Diagram)	6-7
6.	Backend Implementation	8-9
7.	Frontend Implementation	10-11
8.	SSRS Integration	12
9.	API Interaction	12
10.	Reporting & Analytics (Power BI)	13
11.	Azure Integration	14
12.	Results and Outcomes	14-15
13.	Conclusion	15

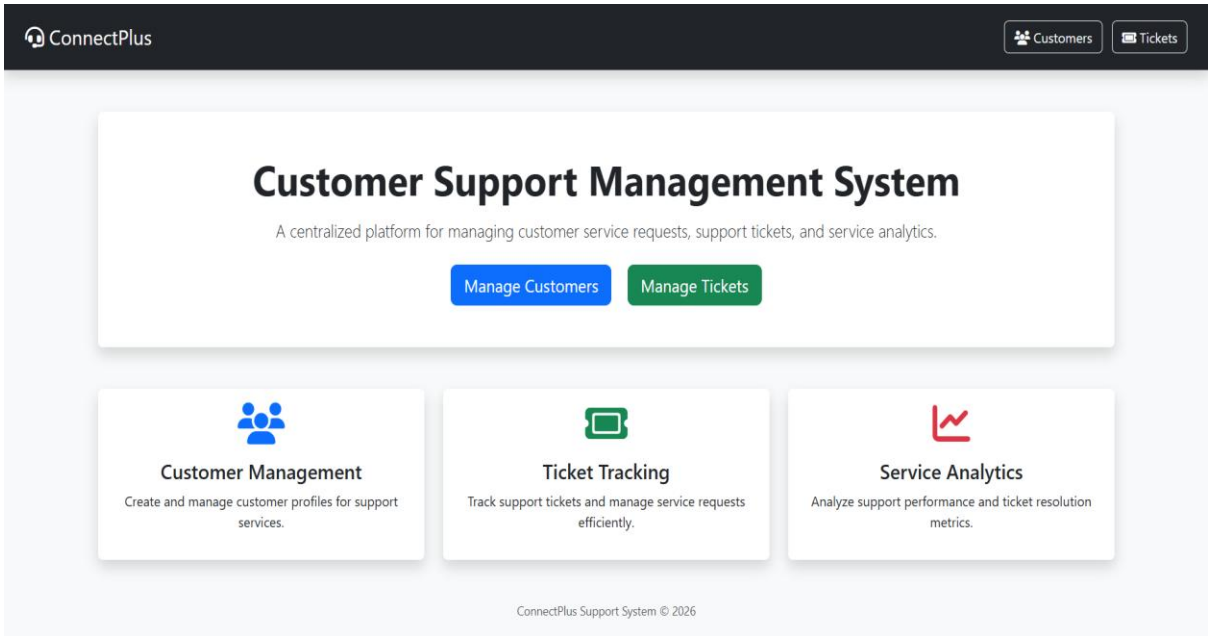
1. Introduction

The ConnectPlus Support System is a web-based application designed to manage customer service operations efficiently. The system enables organizations to handle customer queries, create and track service tickets, and analyze service performance.

In modern business environments, customer support is critical for maintaining customer satisfaction and operational efficiency. This system provides a centralized platform where support agents can manage customer interactions, assign tickets, and monitor progress.

The application integrates frontend, backend, database, and analytics components into a unified system, ensuring seamless operation and scalability.

UI:



2. Objectives

The main objectives of this project are:

- To design a system for managing customer support operations
- To develop a backend API for handling tickets and customers
- To build a responsive frontend interface
- To implement a structured relational database
- To generate insights using Power BI dashboards

This system improves efficiency, reduces manual errors, and supports data-driven decision-making.

3. System Architecture

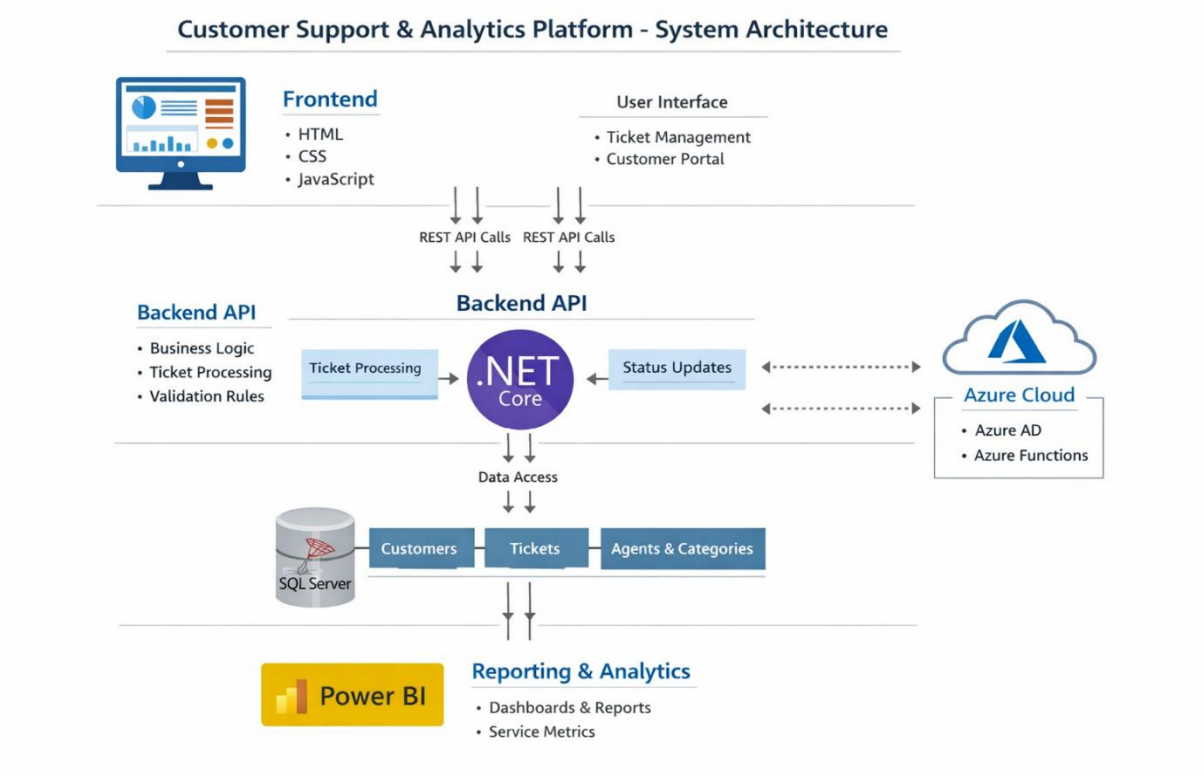
The system follows a three-tier architecture consisting of frontend, backend, and database layers.

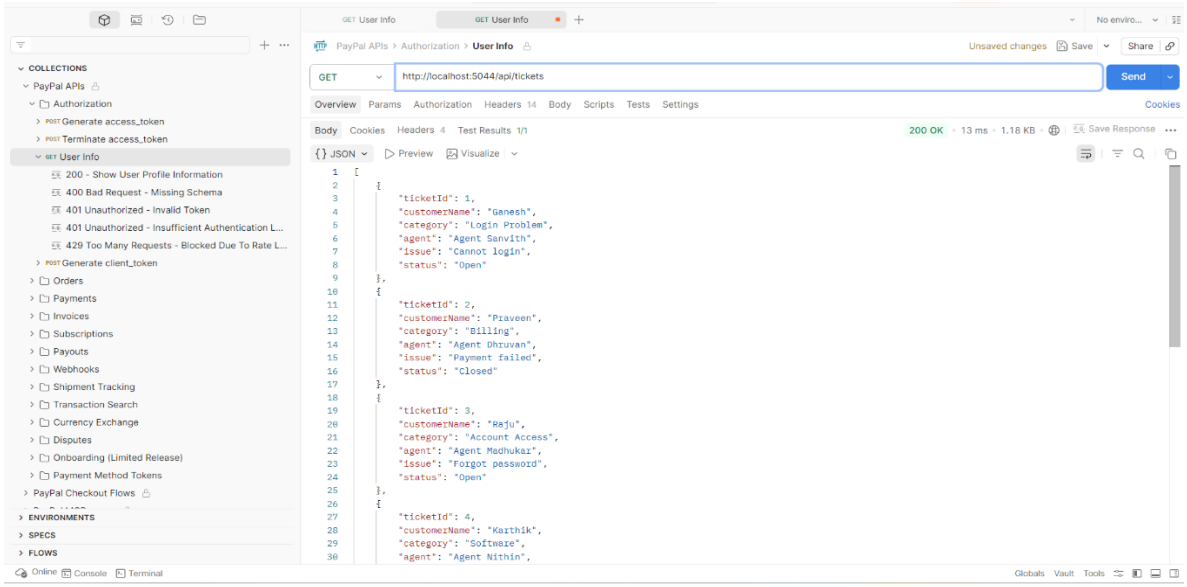
- The frontend (HTML, CSS, JavaScript) provides the user interface for ticket and customer management
- The backend (.NET Core Web API) handles business logic, validation, and API processing
- The SQL Server database stores customer and ticket data
- Power BI is used for reporting and analytics

Additionally, Azure services such as Azure AD and Azure Functions are conceptually included for scalability and security.

This architecture ensures modularity, scalability, and maintainability of the system.

Architecture Diagram:





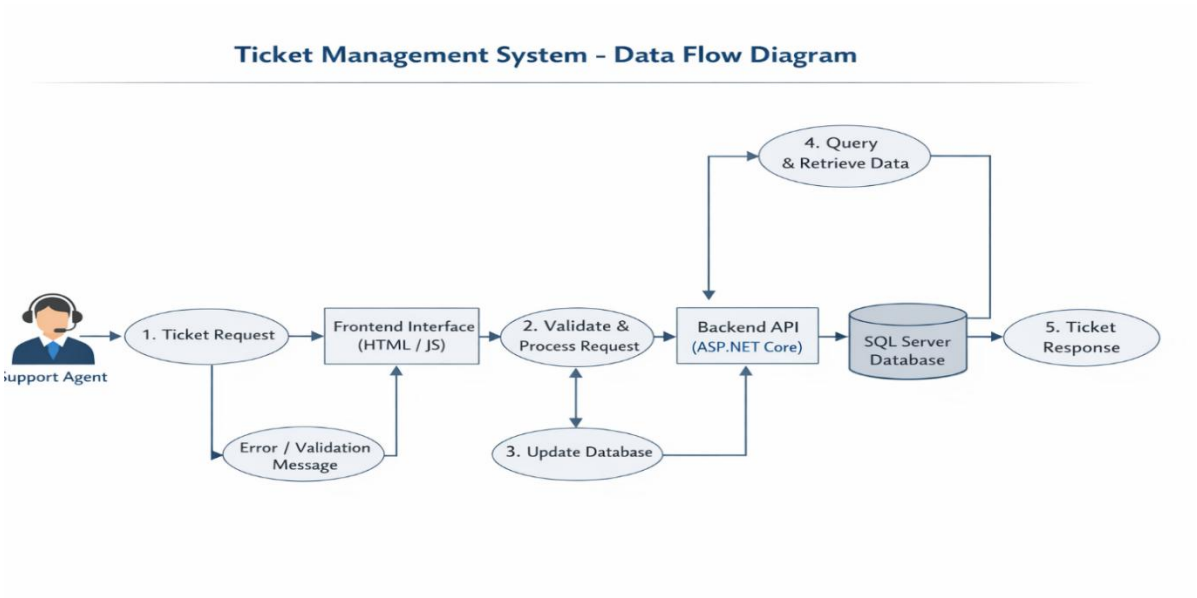
4. Data Flow Diagram (DFD)

The Data Flow Diagram (DFD) illustrates how data flows through the system from user input to final output.

- The user (support agent) submits a ticket request
- The frontend sends the request to the backend API
- The backend validates and processes the request
- The database stores and retrieves data
- The processed response is returned to the user

This flow ensures efficient communication between components and proper handling of service requests.

DFD Diagram:



5. Database Design (ER Diagram)

The database is designed using a normalized relational schema to ensure data integrity and reduce redundancy.

Tables:

- Customers
- Agents
- TicketCategories
- Tickets

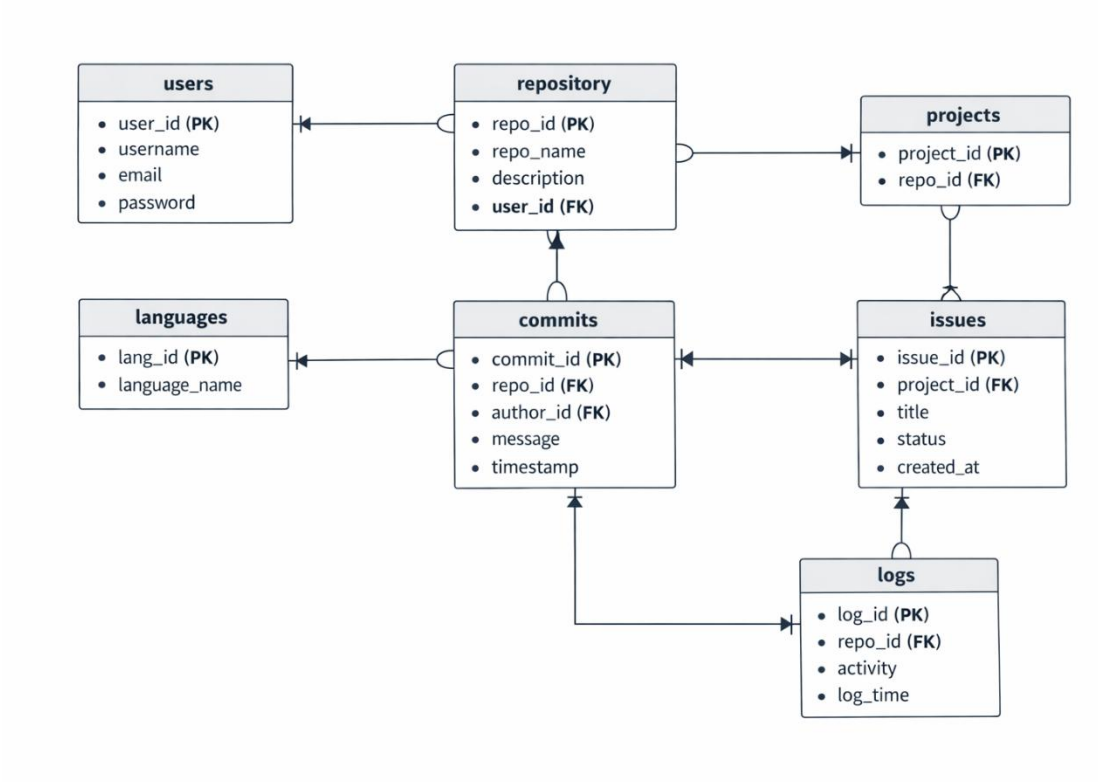
Relationships:

- One Customer can have multiple Tickets
- One Agent can handle multiple Tickets
- One Category can be associated with multiple Tickets

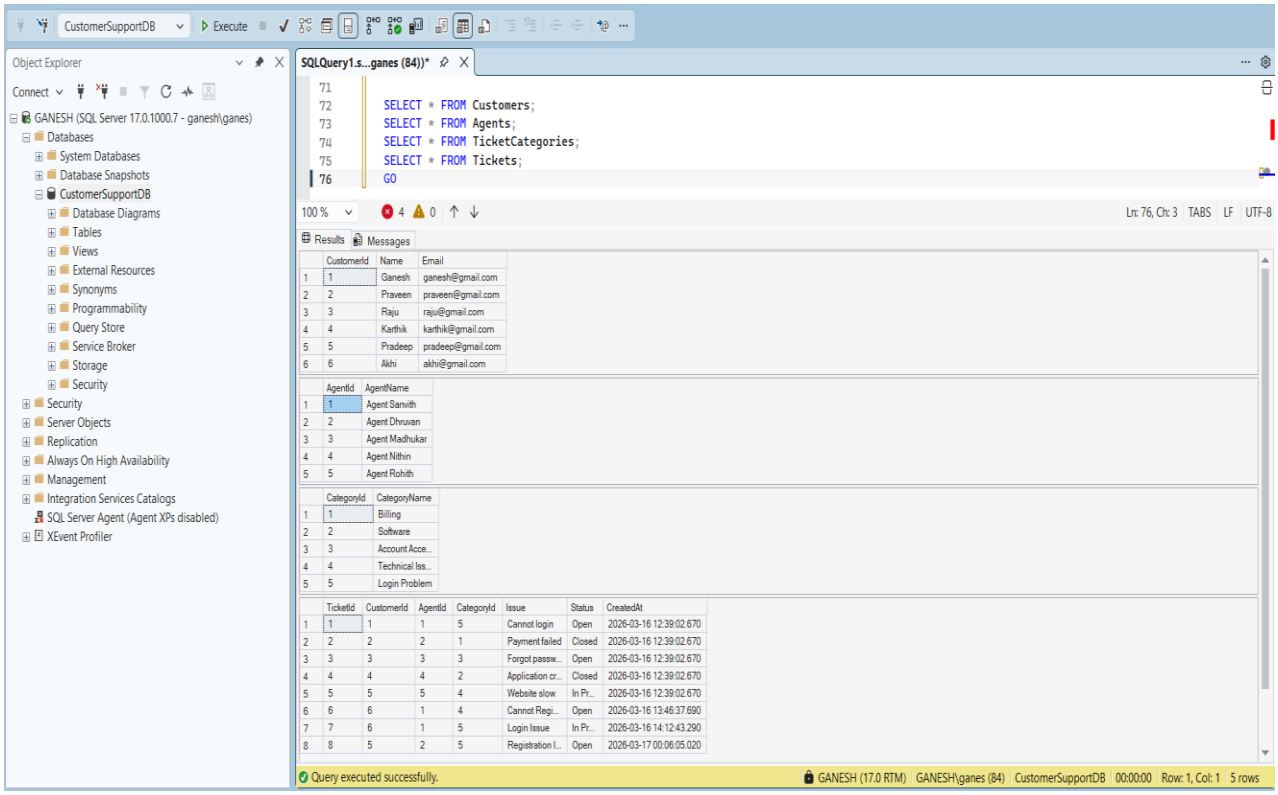
Primary and foreign keys are used to maintain relationships between tables.

This design supports efficient querying and ensures consistency in data storage.

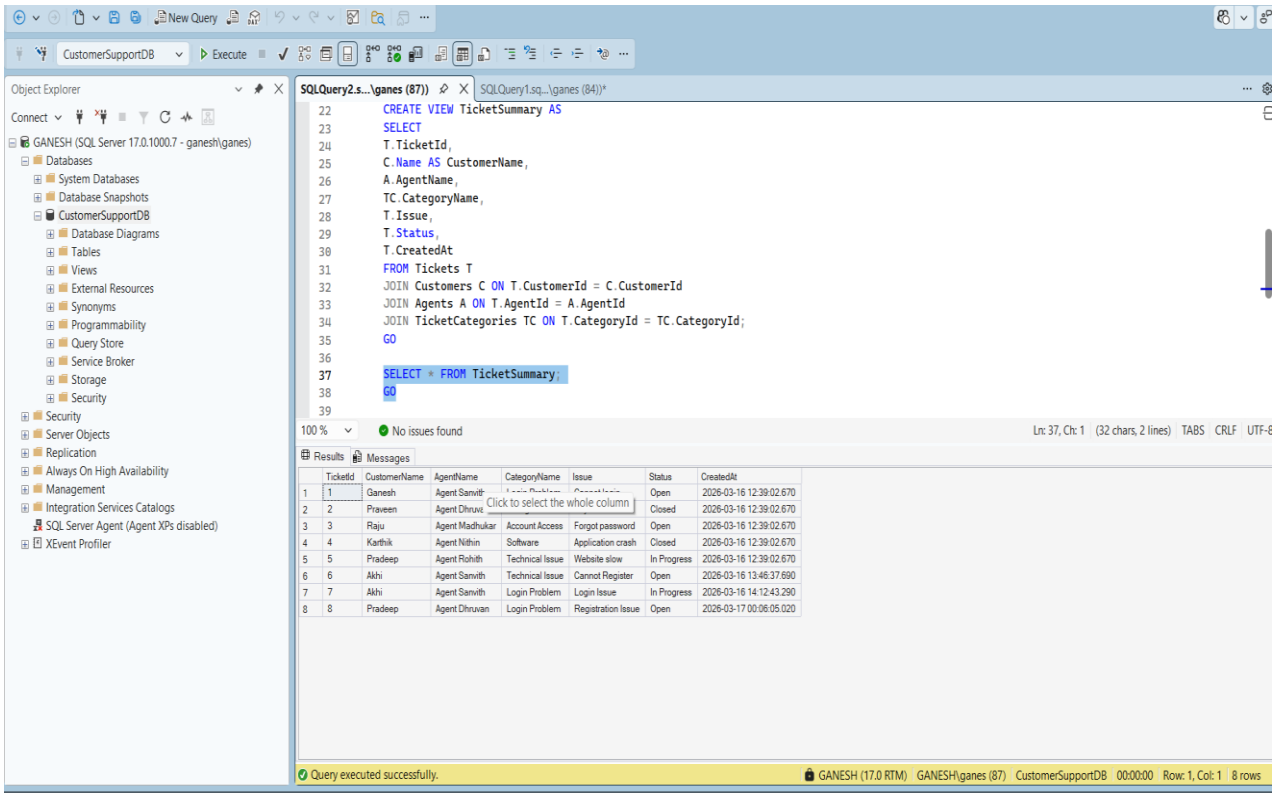
ER Diagram:



SSMS tables view:



TicketSummary view:



6. Backend Implementation

The backend is developed using ASP.NET Core Web API, which provides RESTful services for handling customer and ticket data.

Key Features:

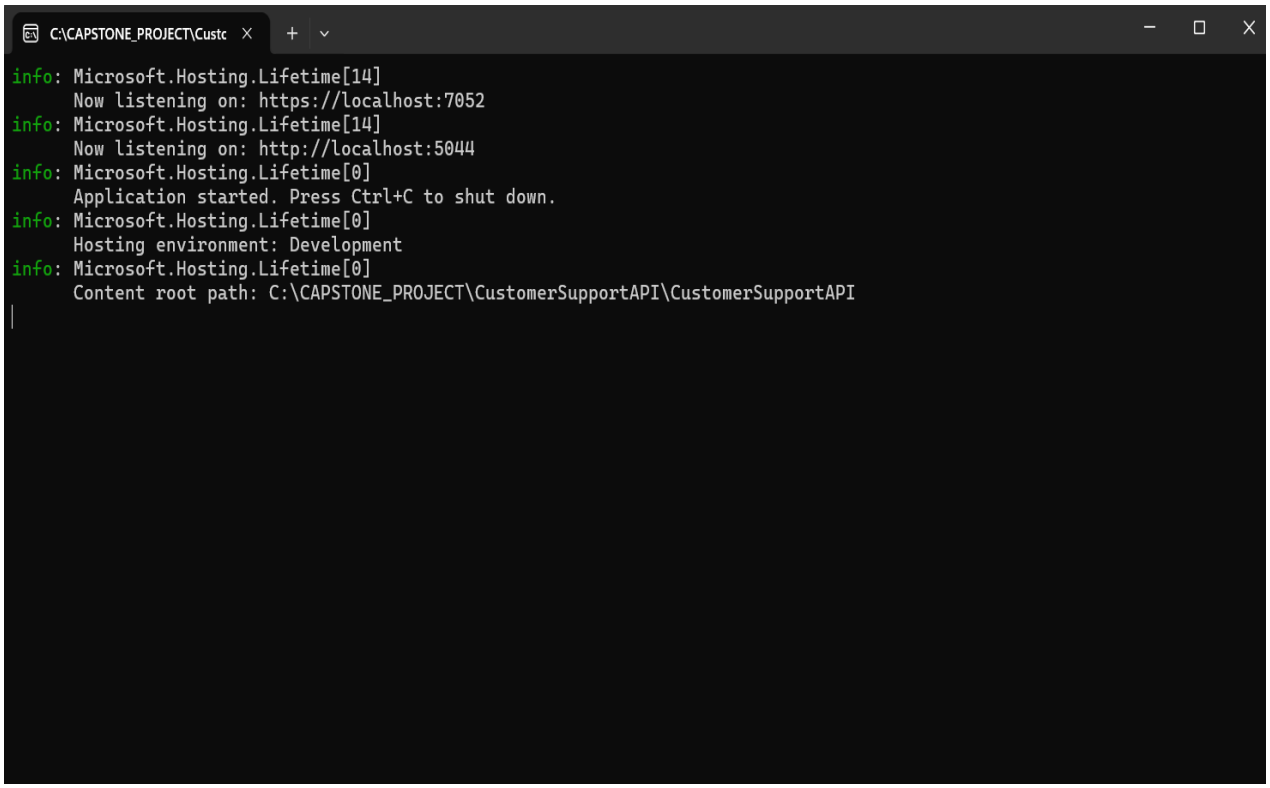
- Create and retrieve customers
- Create and retrieve tickets
- Update ticket status
- Implement validation rules

Business Rules:

- Duplicate customers are not allowed
- Duplicate tickets are prevented
- Ticket issue cannot be empty
- Invalid status transitions are restricted

Exception handling ensures that errors are returned as proper API responses.

This backend structure ensures scalability, maintainability, and secure data processing.



Validation Logic:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Data.SqlClient;
using CustomerSupportAPI.Models;
namespace CustomerSupportAPI.Controllers
{
    [ApiController]
    [Route("api/customers")]
    0 references
    public class CustomerController : ControllerBase
    {
        private string connectionString =
        "Server=GANESH;Database=CustomerSupportDB;Trusted_Connection=True;TrustServerCertificate=True;";
        [HttpPost]
        0 references
        public IActionResult AddCustomer([FromBody] Customer customer)
        {
            using (SqlConnection conn = new SqlConnection(connectionString))
            {
                conn.Open();
                string checkQuery =
                "SELECT COUNT(*) FROM Customers WHERE Email=@email";
                SqlCommand checkCmd = new SqlCommand(checkQuery, conn);
                checkCmd.Parameters.AddWithValue("@email", customer.Email);
                int count = (int)checkCmd.ExecuteScalar();
                if (count > 0)
                {
                    return BadRequest("Customer already exists");
                }
                string query =
                "INSERT INTO Customers(Name,Email) VALUES(@name,@email)";
                SqlCommand cmd = new SqlCommand(query, conn);
                cmd.Parameters.AddWithValue("@name", customer.Name);
                cmd.Parameters.AddWithValue("@email", customer.Email);
                cmd.ExecuteNonQuery();
```

```
                }
                string query =
                "INSERT INTO Customers(Name,Email) VALUES(@name,@email)";
                SqlCommand cmd = new SqlCommand(query, conn);
                cmd.Parameters.AddWithValue("@name", customer.Name);
                cmd.Parameters.AddWithValue("@email", customer.Email);
                cmd.ExecuteNonQuery();
            }
            return Ok("Customer Added Successfully");
        }
        [HttpGet]
        0 references
        public IActionResult GetCustomers()
        {
            List<Customer> customers = new List<Customer>();
            using (SqlConnection conn = new SqlConnection(connectionString))
            {
                conn.Open();
                string query = "SELECT Name,Email FROM Customers";
                SqlCommand cmd = new SqlCommand(query, conn);
                SqlDataReader reader = cmd.ExecuteReader();
                while (reader.Read())
                {
                    customers.Add(new Customer
                    {
                        Name = reader["Name"].ToString(),
                        Email = reader["Email"].ToString()
                    });
                }
            }
            return Ok(customers);
        }
    }
}
```

7. Frontend Implementation

The frontend is built using HTML, CSS, Bootstrap, and JavaScript.

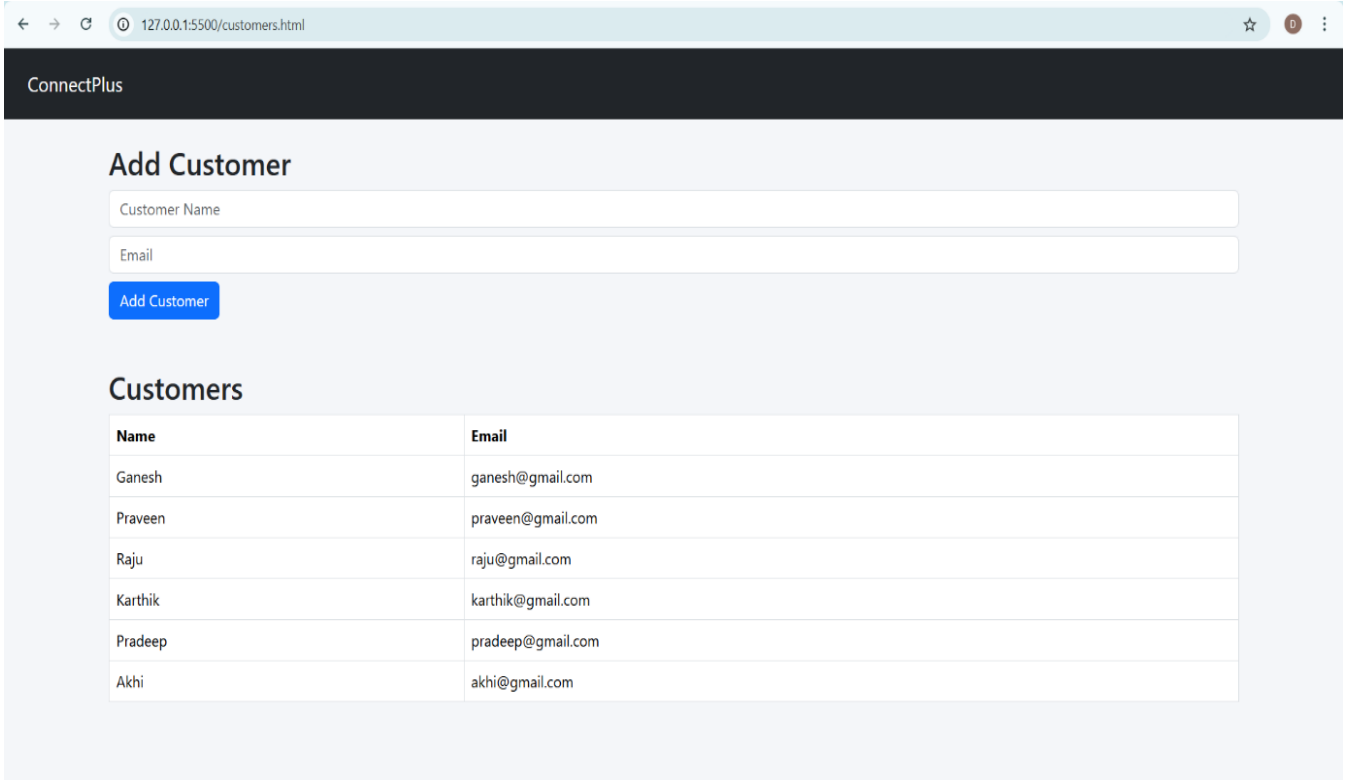
Features:

- Customer management page
- Ticket creation form
- Ticket list table
- Status update dropdown
- Responsive UI design

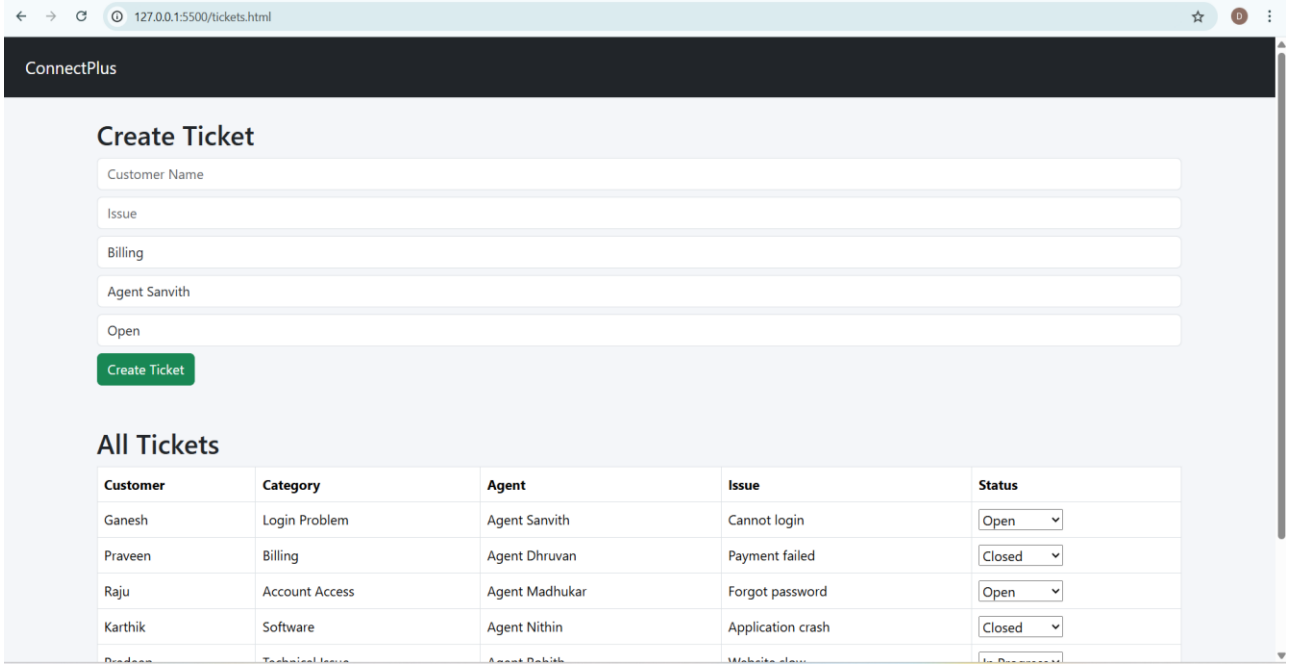
JavaScript Fetch API is used to communicate with the backend and dynamically update the UI without page reloads.

This improves user experience and ensures smooth interaction.

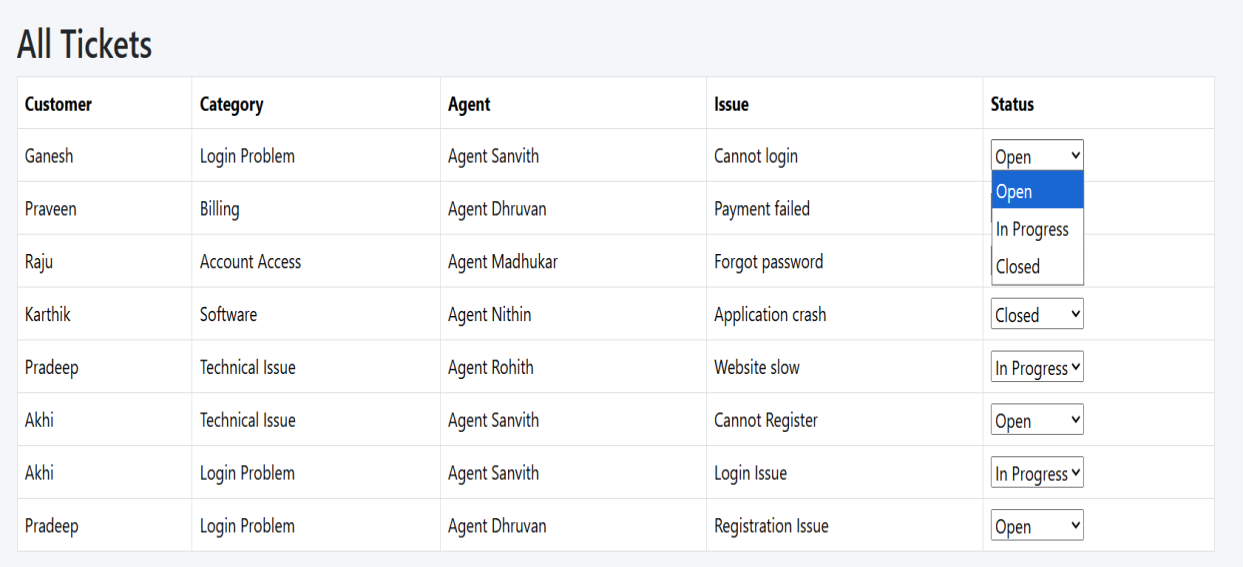
Customers Page:



Tickets Page:



Status Update:



SPA (Single Page Application):

The frontend follows SPA (Single Page Application) concepts using JavaScript to dynamically update content without full page reloads.

Instead of reloading pages, API calls are made using the Fetch API to update ticket and customer data in real time. This improves performance and provides a smoother user experience.

DOM manipulation is used to render updated data in tables after operations such as ticket creation or status updates. SPA architecture reduces server load and enhances responsiveness of the application.

8. SSRS Integration

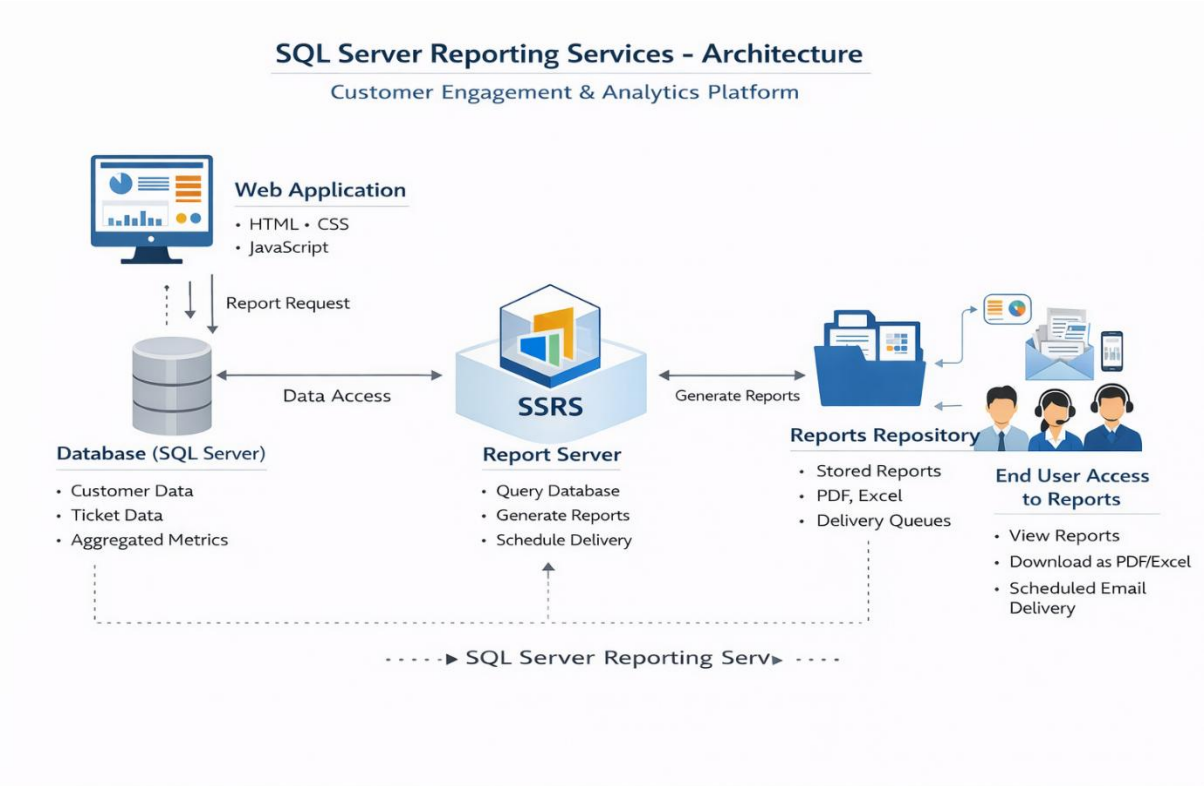
SSRS (SQL Server Reporting Services) is used to generate structured reports from the database.

The frontend can access SSRS reports through embedded links or report viewers. These reports provide detailed information about ticket status, agent performance, and service metrics.

SSRS helps in generating printable and formatted reports required for operational and management purposes.

It complements Power BI by providing static, structured reports.

SSRS Architecture:



9. API Interaction

The frontend communicates with the backend using RESTful APIs. JavaScript Fetch API is used to send HTTP requests and receive responses.

Eg: fetch("http://localhost:5044/api/tickets")

This allows the frontend to perform operations such as creating tickets, retrieving data, and updating status dynamically.

10. Reporting & Analytics (Power BI)

Power BI is used to visualize service data and generate insights.

KPIs:

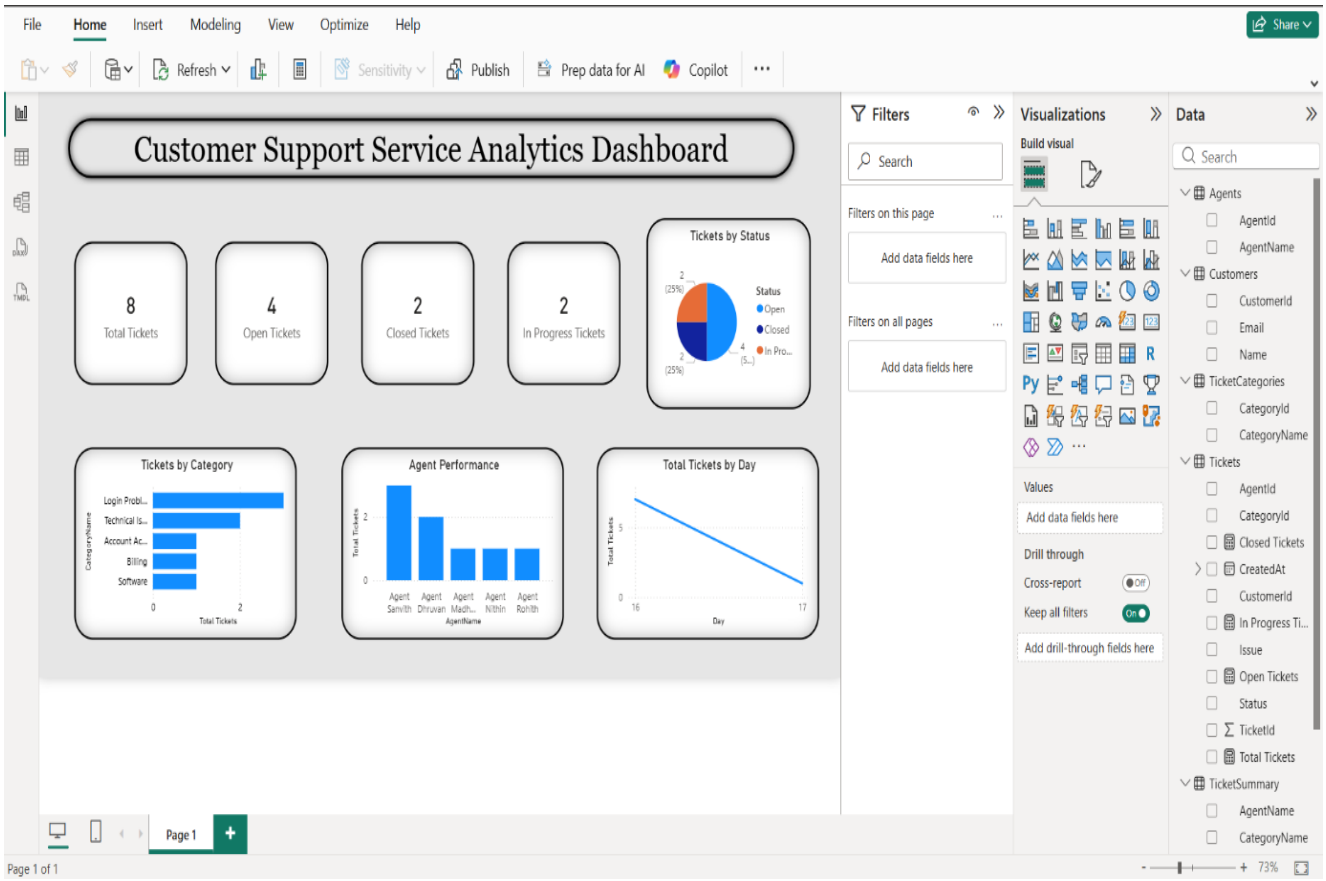
- Total Tickets
- Open Tickets
- Closed Tickets
- In Progress Tickets

Visuals:

- Pie chart (ticket status)
- Bar chart (category-wise tickets)
- Column chart (agent performance)
- Line chart (ticket trend over time)

These insights help management improve service performance and decision-making.

Power BI Dashboard:



11.Azure Integration

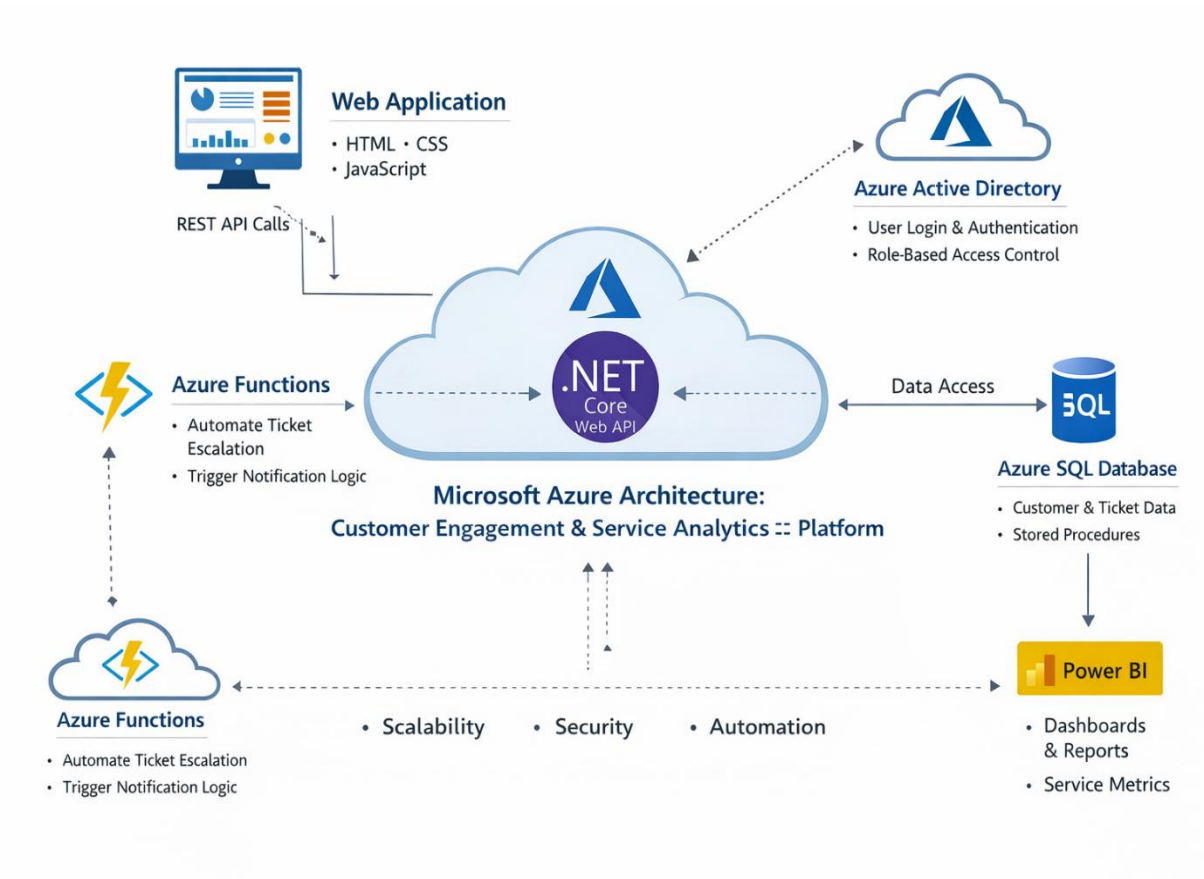
Azure services are conceptually integrated to enhance system scalability, security, and automation capabilities. These services allow the system to be extended into a cloud-based architecture.

Azure Active Directory (Azure AD) can be used for authentication and role-based access control. It ensures that only authorized users such as support agents and administrators can access the system.

Azure Functions can be used to implement serverless workflows such as automatic ticket escalation when service level agreements (SLAs) are not met.

This integration makes the system cloud-ready and suitable for enterprise-level deployment.

Azure Architecture:



12. Results and Outcomes:

The ConnectPlus Support System was successfully implemented as an end-to-end solution for managing customer support operations.

The system integrates frontend, backend, database, and analytics components to provide efficient ticket handling and monitoring.

Key Outcomes:

- Efficient creation and tracking of service tickets
- Real-time status updates using API integration
- Prevention of duplicate and invalid data through validation logic
- Structured database with proper relationships
- Improved data visibility using Power BI dashboards

Observations:

- Most tickets can be tracked easily based on status
- Agents can handle requests more efficiently
- Data-driven insights help in identifying trends

The system improves operational efficiency and ensures better customer service management.

13. Conclusion

The project successfully demonstrates the development of a customer engagement and service analytics platform using modern technologies.

It follows a structured approach by integrating frontend, backend, database, and reporting components into a unified system.

Key Highlights:

- Developed RESTful APIs using ASP.NET Core
- Designed normalized database using SQL Server
- Implemented responsive frontend using HTML, CSS, and JavaScript
- Built Power BI dashboard for analytics and insights

Benefits:

- Faster ticket processing and management
- Improved data accuracy and validation
- Better decision-making through analytics
- Scalable and maintainable system design